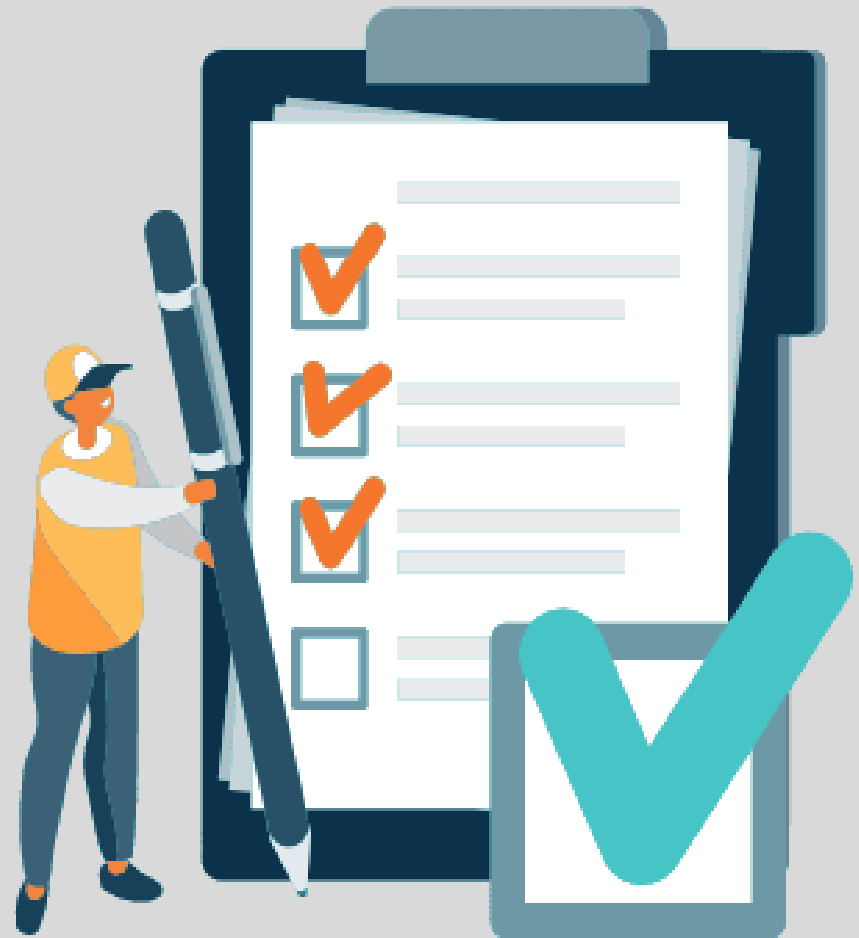
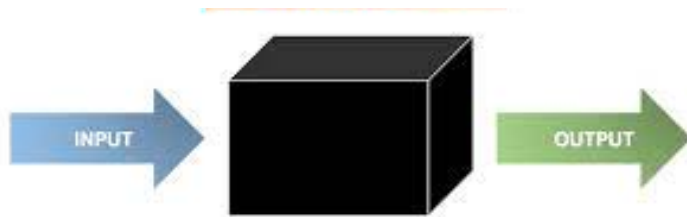


Types of Software Testing



Black box Testing



- Black box testing is a software testing style wherein the objective is to examine whether software works for end-users as intended without worrying about the internal system.
- In this, a tester observes the behaviour of a system entirely by inputs and outputs.

Black Box Testing Techniques

- **Equivalence partitioning:** the inputs are divided into different groups or partitions, and one input from every group is tested to find the results.
- **Boundary value analysis:** the tests in this technique are designed to include representatives of boundary values (inner and outer limits) in a range.
- **Decision table testing:** This technique helps test the system's behaviour for different input combinations. the inputs are represented in tabular form.

equivalence partitioning

AGE

Enter Age

*Accepts value 18 to 56

EQUIVALENCE PARTITIONING		
Invalid	Valid	Invalid
≤ 17	18-56	≥ 57

Boundary value analysis

For 1 Variable

Boundary
Value Analysis

The range of x1: 10 to 90.

<https://t4tutorials.com/>

	X1
Min	10
Min+	11
Nominal	50
Max-	89
Max	90

Test Case#	X1
1	10
2	11
3	50
4	89
5	90

Total Test cases = (total variables * 4 + 1)

$$5 = 1*4+1$$

4 for (Min, Min+, Max-, Max)

1 for Nominal

Boundary Value Analysis

The range of x1: 10 to 90.

The range of x2: 20 to 70.

	X1	X2
Min	10	20
Min+	11	21
Nominal	50	45
Max-	89	69
Max	90	70

<https://t4tutorials.com/>

For 2 Variable

Total Test cases = (total variables * 4 + 1)

$$9 = 2 * 4 + 1$$

4 for (Min, Min+, Max-, Max)

1 for Nominal

Test Case #	X1,X2	Test Case #	X1,X2	Test Case #	X1,X2
1	10,45	2	11,45	3	50,45
4	89,45	5	90,45	6	50,20
7	50,21	8	50,69	9	50,70

decision table testing

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username (T/F)	F	T	F	T
Password (T/F)	F	F	T	T
Output (E/H)	E	E	E	H

Legend:

- T - Correct username/password
- F - Wrong username/password
- E - Error message is displayed
- H - Home screen is displayed

- A Decision Table is the method used to build a complete set of test cases without using the internal structure of the program in question.
- In order to create test cases we use a table to contain the input and output values of a program.
- **Example 1: How to make Decision Base Table for Login Screen**

decision table testing

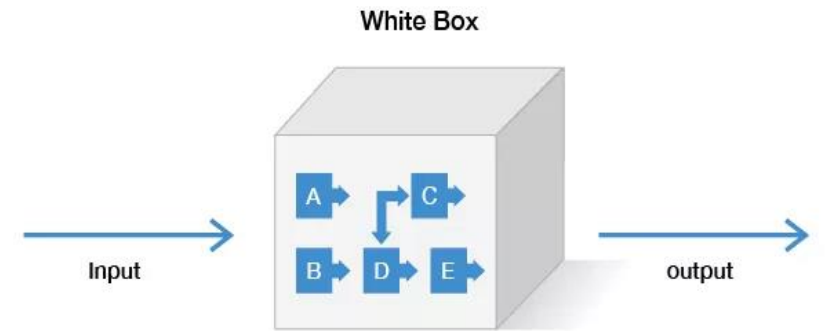
- **Example 2: How to make Decision Table for Upload Screen**
- Now consider a dialogue box that will ask the user to upload a photo with certain conditions like –
 - 1.You can upload only '.jpg' format image
 - 2.file size less than 32kb
 - 3.resolution 137*177.

decision table testing

Conditions	Case 1	Case 2	Case 3	Case 4	Case 5	Case 6	Case 7	Case 8
Format	.jpg	.jpg	.jpg	.jpg	Not .jpg	Not .jpg	Not .jpg	Not .jpg
Size	Less than 32kb	Less than 32kb	>= 32kb	>= 32kb	Less than 32kb	Less than 32kb	>= 32kb	>= 32kb
resolution	137*177	Not 137*177	137*177	Not 137*177	137*177	Not 137*177	137*177	Not 137*177
Output	Photo uploaded	Error message resolution mismatch	Error message size mismatch	Error message size and resolution mismatch	Error message for format mismatch	Error message format and resolution mismatch	Error message for format and size mismatch	Error message for format, size, and resolution mismatch

White Box testing

- White box testing is a form of application testing that provides the tester with complete knowledge of the application being tested, including access to source code and design documents.
- This in-depth visibility makes it possible for white box testing to identify issues that are invisible to black box testing.



White Box Testing Techniques

- **Statement coverage:** makes sure that all executable statements in the code are run and tested at least once.
- **Branch coverage:** the code is mapped into the branches of conditional logic and ensures that unit tests cover every branch.
- **Path coverage:** the tester writes unit tests to execute all the possible paths through the program's control flow in order to identify broken, redundant, and inefficient paths.

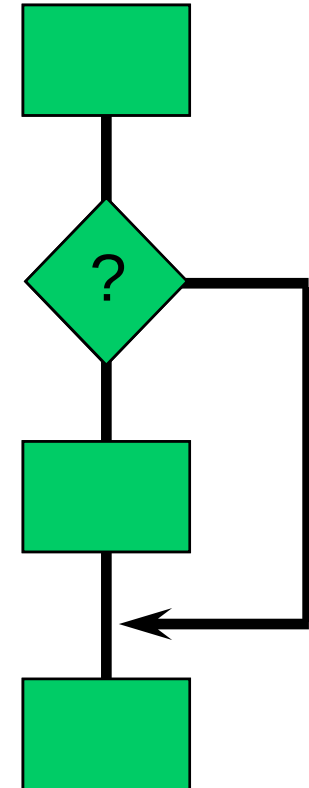
Statement coverage

Statement coverage is normally measured by a software tool.

- percentage of executable statements exercised by a test suite

$$\text{Statement Coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} \times 100$$

- example:
 - program has 100 statements
 - tests exercise 87 statements
 - statement coverage = 87%



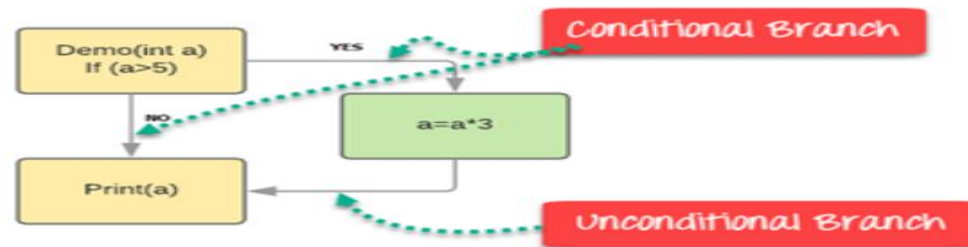
Typical ad hoc testing achieves 60 - 75%

Branch Coverage

- The purpose of branch coverage is to ensure that each decision condition from every branch is executed at least once.

$$\text{Branch Coverage} = \frac{\text{Number of Executed Branches}}{\text{Total Number of Branches}}$$

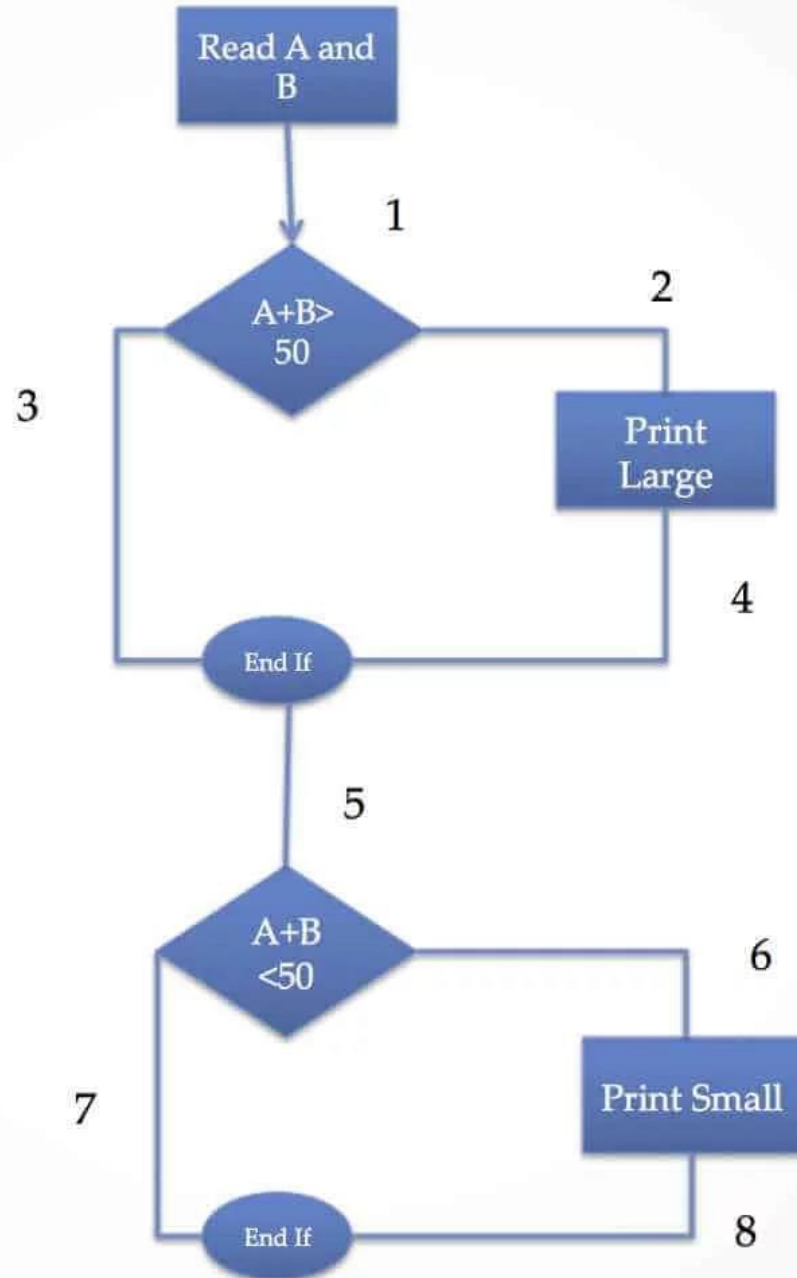
```
Demo(int a) {  
    If (a > 5)  
        a = a * 3  
    Print (a)  
}
```



Branch Coverage will consider unconditional branch as well

Test Case	Value of A	Output	Branch Coverage
1	2	2	33%
2	6	18	67%

Path coverage



- Path Coverage ensures covering of all the paths from start to end.

- All possible paths are-

1-3-5-7

1-3-5-6-8

1-2-4-5-6-8

1-2-4-5-7

PathCoverage = Number of Paths Covered/ Number of Total Paths

Black box testing vs. White Box testing

Black Box Testing	White Box Testing
It is also called Specification Based Technique.	It is also called Structural Testing Technique.
Internal structure and coding knowledge is not required.	Internal structure and coding knowledge is required.
Main concentrate on functionality of system.	Main concentrate on code structure ,branches , loops, conditions etc.
Implementation knowledge is not required.	Implementation knowledge is required.

Integration Testing

- Integration testing is carried out **based on global or high-level design**.
- Integration testing is the process of testing how different modules or components of your software work together.
- Integration testing is performed to expose the defects in the interfaces between components, interactions of different parts or systems such as an Operating System and File system, etc. Here the main focus is on how other modules or components' interaction works, not the functionality of the individual component.

Objective of Integration Testing

Improve software quality: This testing helps improve your software's overall quality. This leads to a higher quality product that meets the needs of your users.

Reduce development time and cost: Integration tests can help to reduce development time and cost by catching issues early in the development process.

Integration Testing Techniques

- 1. Top-down testing:** is an iterative strategy that connects two or more logically linked components. This approach aims to identify any integration issues that may arise as the different components are combined. It begins with testing the largest or main component of an application. (e.g. to test a complicated online site by starting with the main webpage and progressing through the other components in decreasing priority order.)
- 2. Bottom-up testing:** in this testing, integrated code modules are tested before evaluating the entire system or code base. It begins with testing the smallest components of an application.
- 3. Big-bang testing:** All the components are interlinked as a single system and is tested as a complete system; testing a system with all units linked together is difficult due to the lack of attention paid to checking the interfaces between separate units.

Unit Testing

- A unit testing is a type of software test that focuses on components of a software product.
- The purpose is to ensure that each unit of software code works as expected.
- A unit can be a function, method, module, object, or other entity in an application's source code.
- The objective of a unit testing is to test an entity in the code, ensure that it is coded correctly with no errors, and that it returns the expected outputs for all relevant inputs.
- Usually performed by the developer.

Integration testing vs. Unit testing



Unit Testing

Performed at any time

Performed by a software developer

White box testing

Detection of defects/ issues is easy

Maintenance and cost is low

Focus on one single module

Execution is fast

Conducted in the first stages of software development



Integration Testing

Carried out after unit testing

Performed by separate teams of testers

Black box testing

Detection of defects/ issues is complex

Maintenance and cost is high

Focuses on two or more modules

Execution is slow

Performed after unit testing and before system testing

Component testing

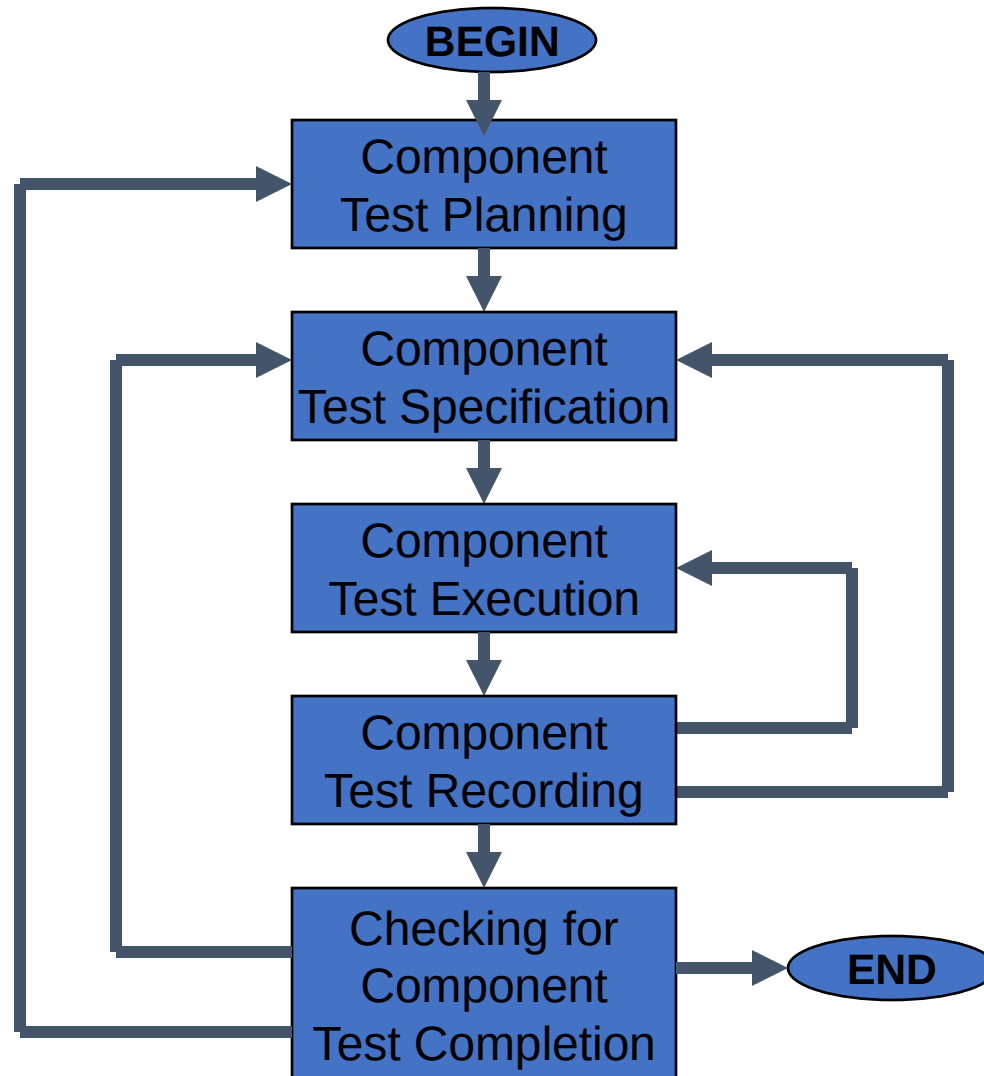
- Component testing is the testing of the different components or modules of a software application independently.
- It is a necessary step that follows unit testing.
- It ensures that each software system component functions correctly and meets the required specifications.
- This type of testing involves individual test objects, such as modules, classes, objects, and programs, without integrating them with other components.

Component Testing Techniques

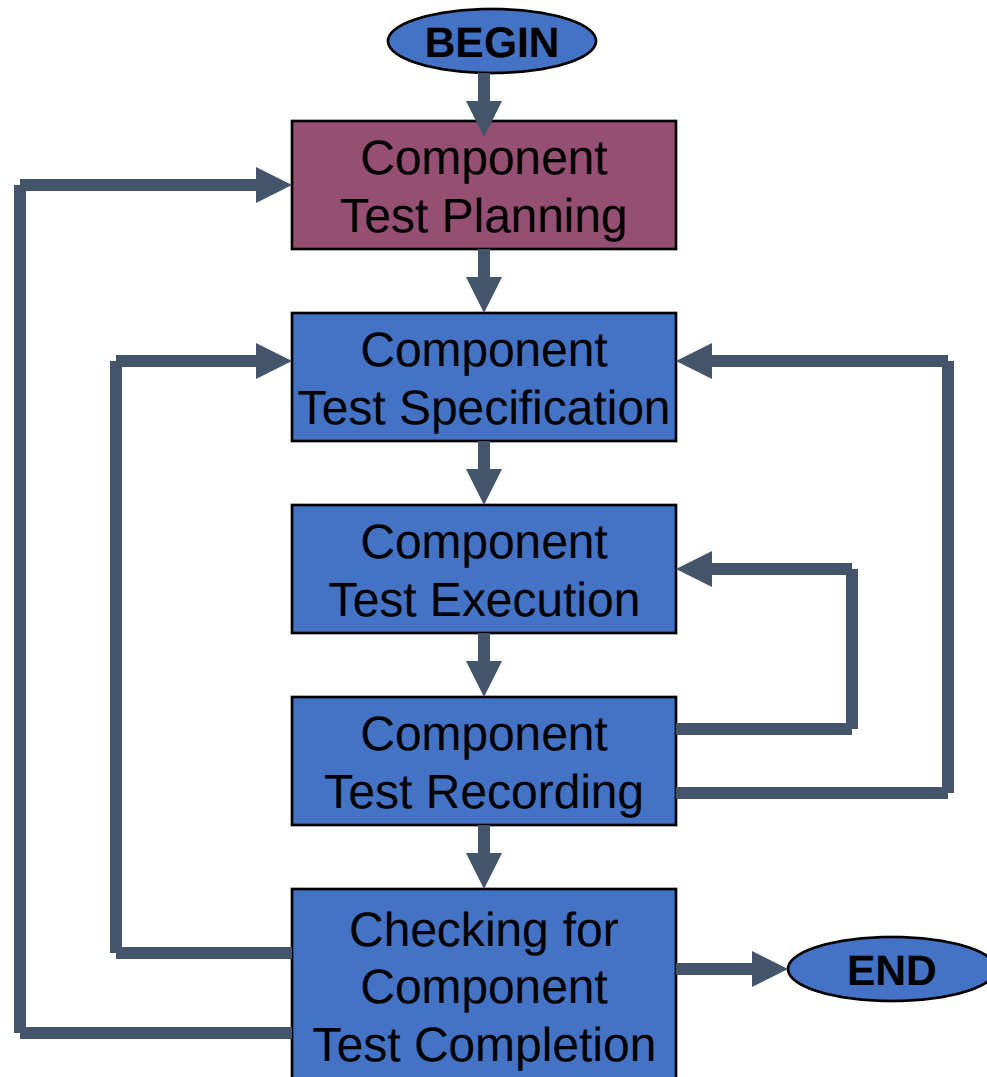
There are two types of Component testing techniques. They are based on the approach developers take to execute Component testing:

1. **Component testing in Small:** Component testing is performed by keeping other components in isolation, it is known as component testing in small (CTIS).
2. **Component testing in Large:** When an individual Component testing is carried out, and other software components are not isolated, it is known as Component testing in large (CTIL).

Component test process



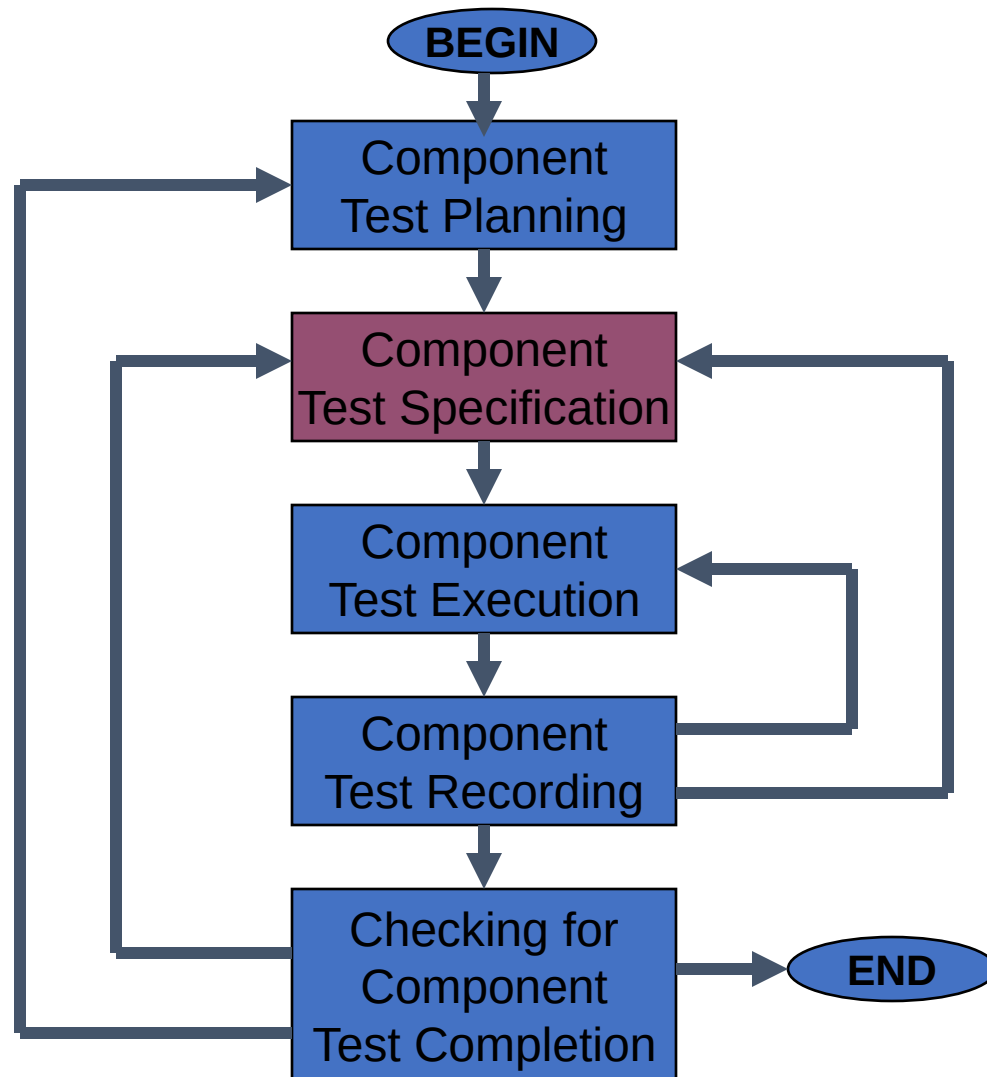
Component test process



Component test planning

- how the test strategy and project test plan apply to the component under test
- any exceptions to the strategy
- all software the component will interact with (e.g. stubs and drivers)

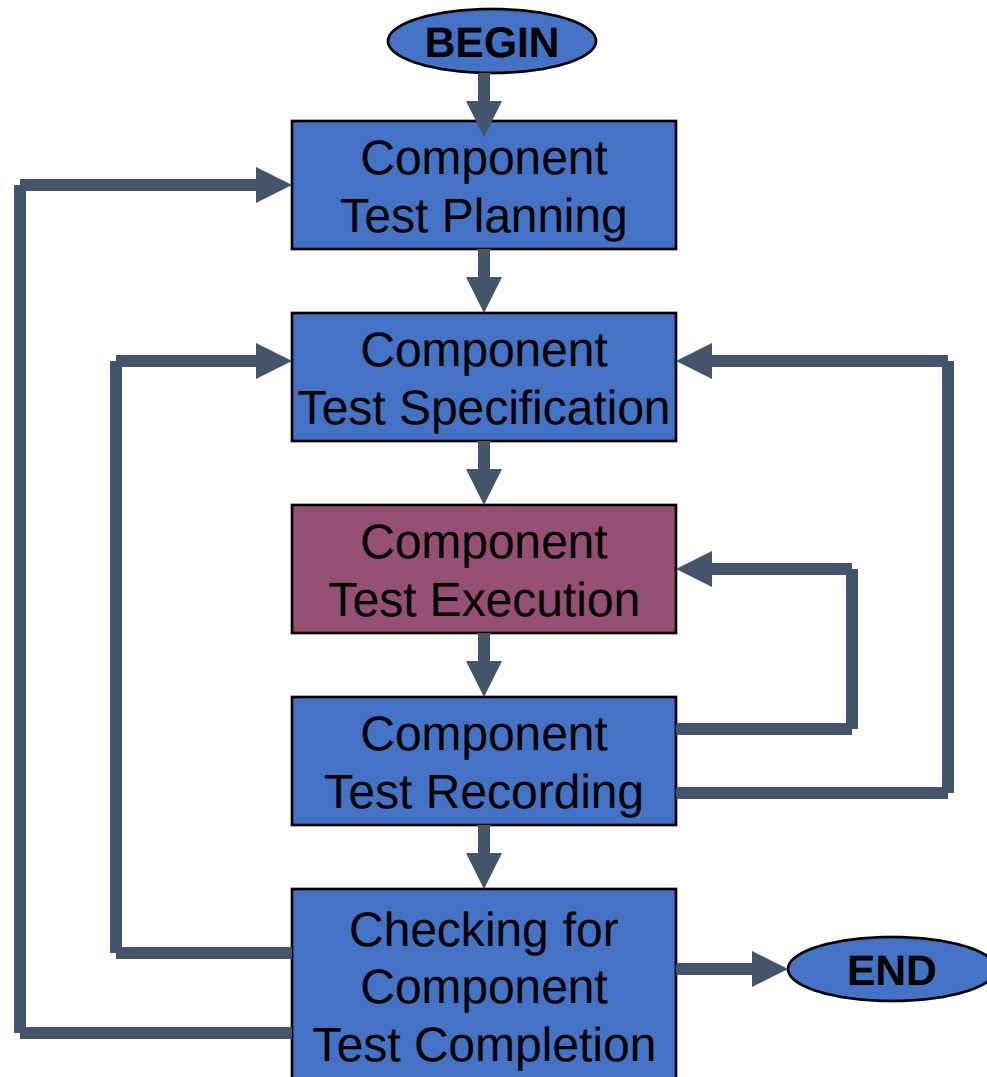
Component test process



Component test specification

- test cases are designed using the test case design techniques specified in the test plan (Section 3)
- Test case:
 - objective
 - initial state of component
 - input
 - expected outcome
- test cases should be repeatable

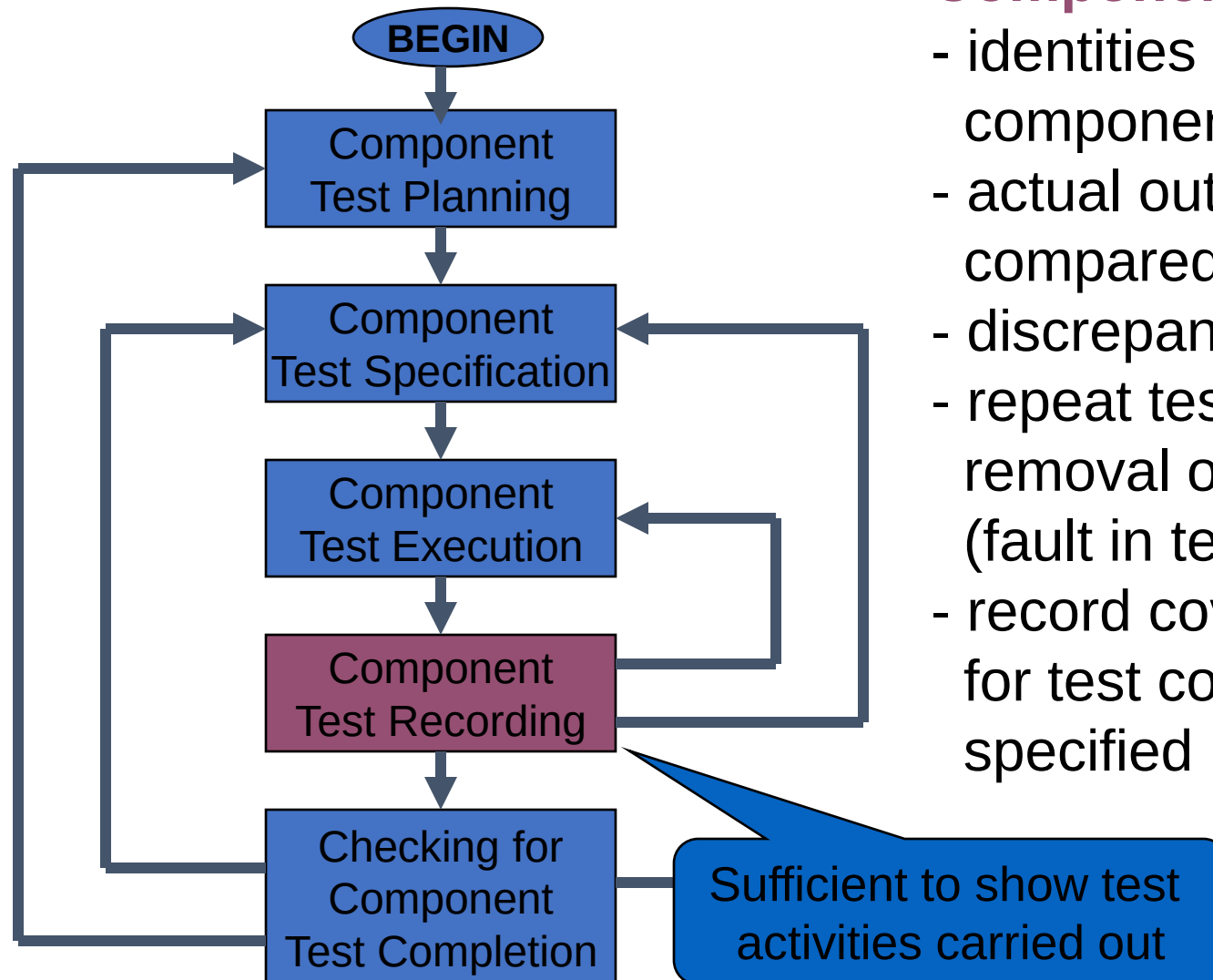
Component test process



Component test execution

- each test case is executed
- standard does not specify whether executed manually or using a test execution tool

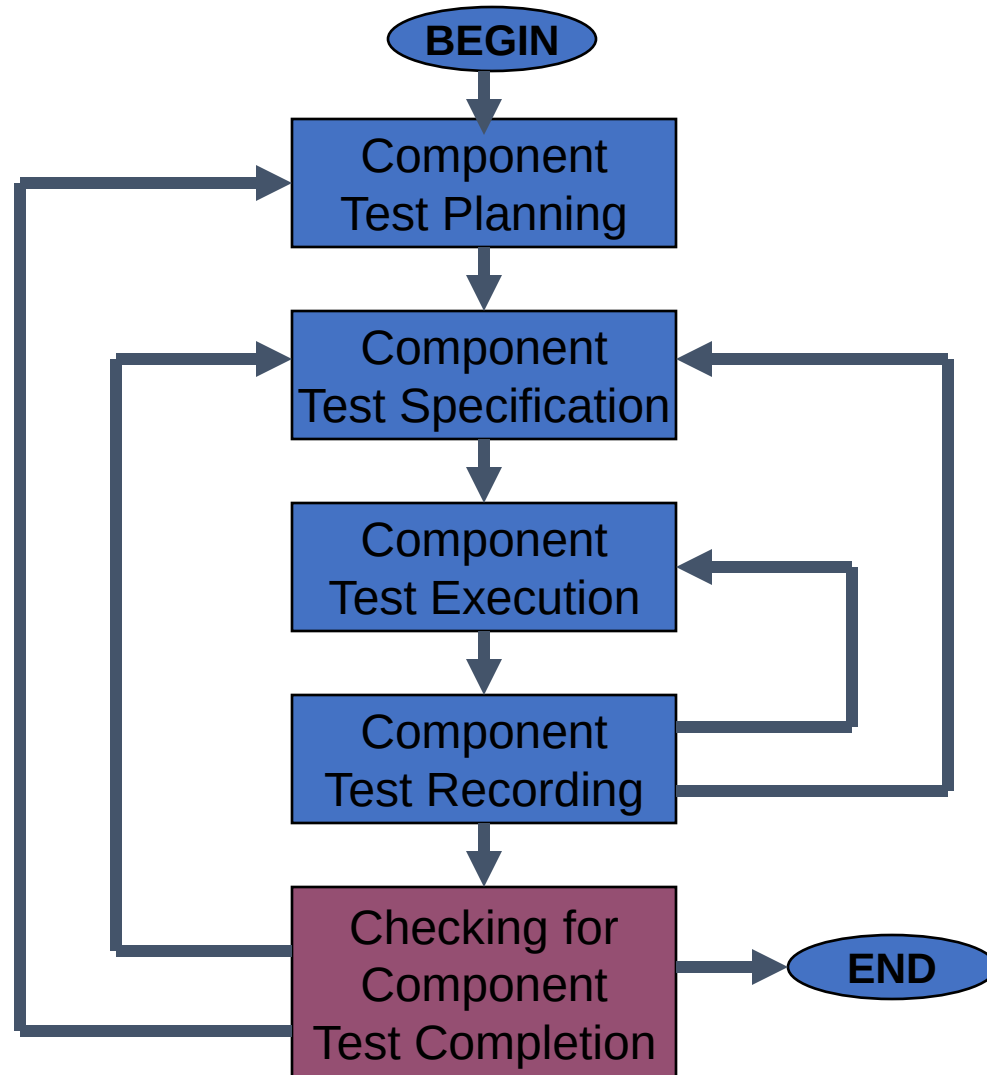
Component test process



Component test recording

- identities & versions of component, test specification
- actual outcome recorded & compared to expected outcome
- discrepancies logged
- repeat test activities to establish removal of the discrepancy (fault in test or verify fix)
- record coverage levels achieved for test completion criteria specified in test plan

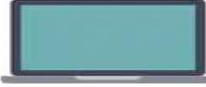
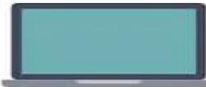
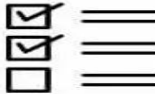
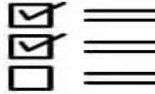
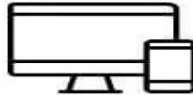
Component test process



Checking for component test completion

- check test records against specified test completion criteria
- if not met, repeat test activities
- may need to repeat test specification to design test cases to meet completion criteria (e.g. white box)

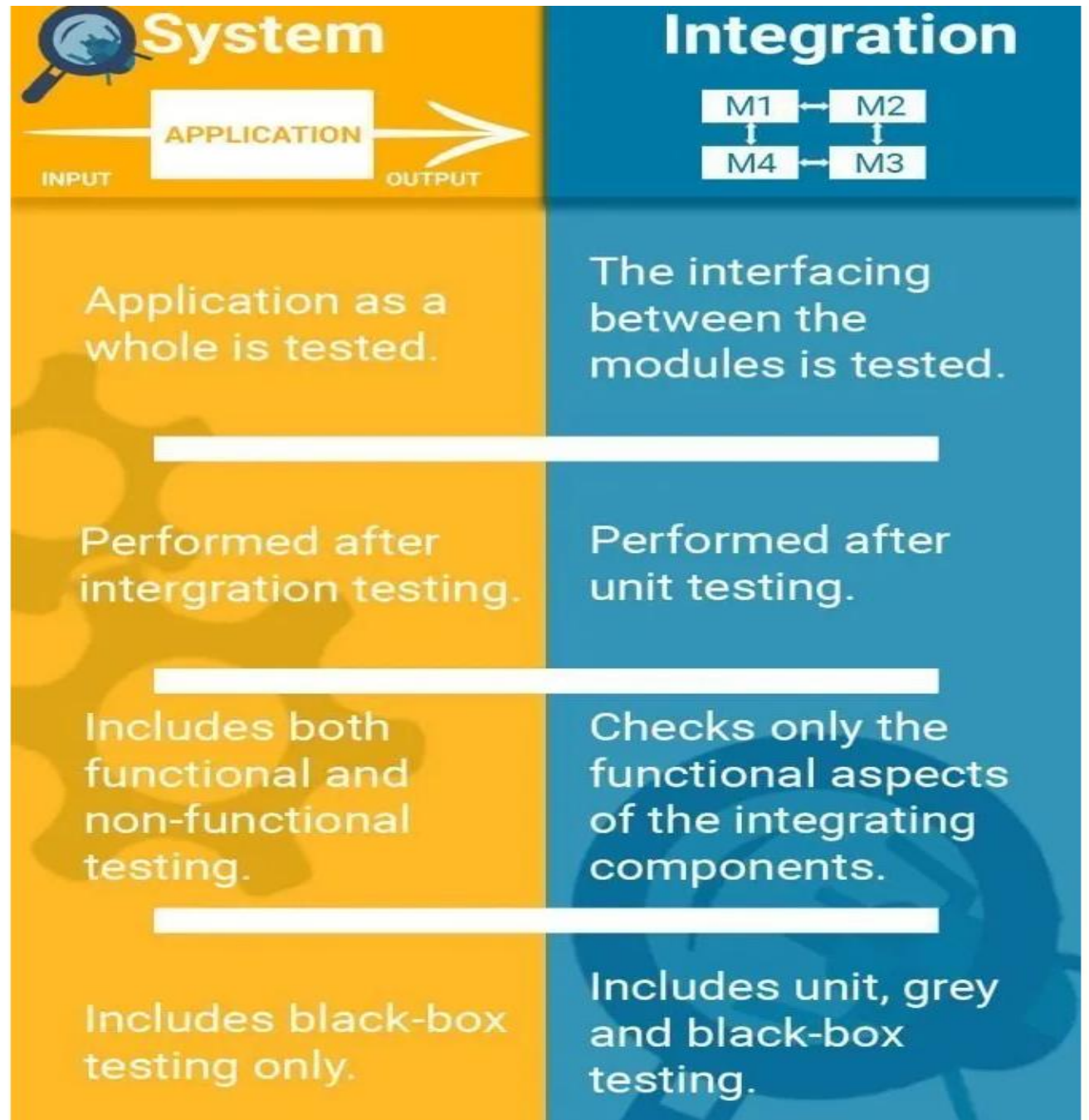
Unit testing vs. Component testing

Component Testing	Unit Testing
 In this testing, each object or component of the software is tested separately.	 In this type of testing, individual modules or programs for program execution.
 It validates use cases and test requirements.	 It is tested against design documents.
 It is performed once the unit testing is performed.	 It is performed before the component testing.
 In component testing, tester do not have knowledge about the internal architecture of the software.	 In Unit testing, developers knows the internal architecture of the software.
 Error detection is bit difficult as compared to unit testing.	 Error detection is easy in unit testing.
 Component testing is performed only after the whole software is developed.	 Unit testing is done after each development step.
 It is a type of black box testing.	 It is a type of white box testing.

System Testing

- System testing tests an integrated system/whole system or product to verify that it meets specified requirements.
- The main goal is to **validate that the delivery system meets the design specifications.**
- The testing team **performs complete or thorough testing in this phase, including system, functional, and non-functional test cases.** Its purpose is to find as many defects as possible and report them.
- System tests are generally executed in different development test environments. However, **the test environments should correspond to the production environment as much as possible to minimize the risk of environment-specific bugs.**

System Testing vs. Integration Testing



Acceptance Testing

- Acceptance testing is associated with the requirements analysis phase.
- Testers **review the Requirement specification and check whether the requirement has enough details to be tested**, there is no ambiguity if non-functional requirements are mentioned as well, whether the requirements have been prioritized, and if product risks have been captured.
- This **determines whether or not the software product/system satisfies the acceptance criteria and enables the users, customers, or other authorized entities to determine whether or not to accept the system.**

The End

